

PROYECTO 2

Sistema Multimodal de Recuperación y Búsqueda

Plan de Trabajo, Distribución de Roles y Estructura del Repositorio

Equipo de 5 integrantes

Contenido

1. Resumen del proyecto	3
2. Aplicaciones a implementar	3
3. Arquitectura unificada y doble rol de PostgreSQL	3
3.1 Las dos rutas de búsqueda	3
3.2 Pipeline común por modalidad.....	4
4. Distribución de roles (5 integrantes).....	5
5. Cronograma y deadlines	5
6. Estructura del repositorio en GitHub	7
7. Convenciones de Git (esto se evalúa).....	8
8. Aclaraciones técnicas clave	8
8.1 SPIMI (obligatorio) vs. BSBI (opcional).....	8
8.2 Docker: “base de datos lista en un comando”	8
8.3 Conexión PostgreSQL ↔ Python	8
9. Entregables finales (checklist).....	9

1. Resumen del proyecto

El objetivo es diseñar e implementar un sistema unificado de recuperación y búsqueda que opere sobre múltiples modalidades (texto, imágenes y audio) bajo una misma arquitectura. El paradigma común es: dividir el contenido en unidades atómicas (split), extraer descriptores adaptados a cada modalidad, construir un diccionario compartido (codebook), generar índices invertidos que mapean palabras/conceptos a chunks, y comparar esta implementación propia contra las técnicas nativas de PostgreSQL (GIN/GiST para texto y pgvector para imagen y audio).

Este documento define qué se construye, cómo se reparte entre los 5 integrantes, en qué plazos, y cómo se organiza el repositorio en GitHub para cumplir con los criterios de evaluación de planificación (división clara de tareas, hitos a tiempo, cierre documentado de issues y equilibrio de contribuciones).

2. Aplicaciones a implementar

El proyecto exige soportar al menos dos de las cuatro aplicaciones sugeridas. La combinación recomendada es:

- **App 1 — Búsqueda Visual E-commerce** (modalidad primaria: imagen). El usuario sube una foto y el sistema retorna productos visualmente similares.
- **App 2 — Búsqueda Musical Inteligente** (modalidad primaria: audio + texto). Buscar canciones por letra (full-text) y por similitud acústica.

Justificación: juntas ejercitan las tres modalidades (texto, imagen y audio), lo que habilita la comparativa más completa de la Fase 3 —GIN/GiST sobre el texto de las letras y pgvector sobre imagen y audio— y demuestra mejor el carácter agnóstico de la arquitectura unificada.

Alternativa de menor alcance: App 1 + App 4 (ambas sobre el Fashion Product Images Dataset, solo texto + imagen, sin audio). Menos código, pero comparativa menos rica.

3. Arquitectura unificada y doble rol de PostgreSQL

Un punto central que el equipo debe tener claro: PostgreSQL cumple DOS papeles simultáneos en el proyecto.

3.1 Las dos rutas de búsqueda

Lado A — Implementación propia (lo que ustedes programan en Python). Índice invertido + codebook + histogramas. Aquí PostgreSQL actúa como almacén pasivo: guarda en tablas las codewords, posting lists e histogramas. La lógica de búsqueda la escriben ustedes.

Lado B — Técnicas nativas de PostgreSQL (los baselines). Aquí PostgreSQL hace el trabajo con sus propios índices: GIN/GiST sobre una columna tsvector para full-text de texto, y pgvector (índice HNSW o IVF) sobre los histogramas almacenados como vectores para imagen y audio.

La comparación consiste en resolver la **misma consulta, sobre los mismos datos, por las dos rutas**, y medir cuál gana en latencia, recall, memoria y accesos a disco. La comparación solo es válida si ambos lados parten de idéntica entrada; garantizarlo es responsabilidad del Lead en el diseño del esquema.

3.2 Pipeline común por modalidad

Etapas	Texto	Imagen	Audio
Split	Párrafos	Patches / regiones	Ventanas 100–200 ms
Extractor	TF-IDF	SIFT	MFCC
Codebook	Top-k palabras (stopwords, stemming)	K-Means → visual words	K-Means → acoustic words
Índice	SPIMI	Histograma de visual words	Histograma de acoustic words
Baseline nativo	GIN / GiST	pgvector (HNSW/IVF)	pgvector (HNSW/IVF)

4. Distribución de roles (5 integrantes)

El corte se hace por modalidad (paralelizable) más roles transversales. El Lead define las interfaces comunes en la Semana 1 para que las tres modalidades avancen sin bloquearse entre sí. Cada ingeniero de modalidad implementa TANTO su versión propia (Lado A) COMO el baseline nativo de su modalidad (Lado B).

#	Rol	Responsabilidades principales	Entregables propios
1	Tech Lead — Arquitectura & DevOps	Define interfaces abstractas comunes (Splitter, Extractor, CodebookBuilder, InvertedIndex). Arma el repo, Docker Compose (PostgreSQL + pgvector) y scripts SQL de init. Integra módulos (pipeline), revisa PRs, administra GitHub Projects e issues, y garantiza que ambos lados (A y B) reciban idéntica entrada.	core/, docker/, esquema SQL, pipeline, integración
2	Ingeniero de Texto	Split por párrafos, TF-IDF, tokenización / stopwords / stemming, codebook lingüístico top-k, índice invertido SPIMI. Comparativa GIN/GiST (full-text).	src/text/, baselines/gin_gist.py
3	Ingeniero de Imágenes	Split por patches, descriptores SIFT, K-Means → visual words, histogramas, índice invertido visual. Comparativa pgvector (HNSW/IVF).	src/image/, parte de baselines/pgvector.py
4	Ingeniero de Audio	Split por ventanas deslizantes, MFCC, K-Means → acoustic words, histogramas, índice invertido acústico. Comparativa pgvector (HNSW/IVF).	src/audio/, parte de baselines/pgvector.py
5	Backend / Apps & Evaluación	API extensible (FastAPI) con los endpoints de las 2 aplicaciones + demo. Marco experimental (cargas 1K/10K/100K), métricas (latencia, recall, memoria, I/O) y gráficos comparativos de la Fase 4.	src/backend/, src/eval/, results/

Transversal a todos: redacción de su sección del informe (README/Wiki), apertura y cierre documentado de sus propios issues, PRs revisados por otro miembro, y participación en la presentación. El Lead programa (no solo administra): su carga de código es comparable a la del resto, manteniendo el equilibrio de contribuciones que evalúa la rúbrica.

5. Cronograma y deadlines

Plan de 6 semanas alineado a las cuatro fases del enunciado. Ajusten las fechas a su calendario real; cada hito corresponde a un milestone en GitHub Projects. Regla práctica: cerrar la Fase 2 antes de tocar las comparativas, porque la Fase 4 concentra gran parte de la nota analítica.

Sem.	Fase	Hito (milestone)	Lidera
1	Fase 1	Requisitos definidos, dataset elegido y documentado, diseño de arquitectura, interfaces comunes escritas, repo + Docker levantando con un comando.	Lead (todos aportan)

Sem.	Fase	Hito (milestone)	Lidera
2	Fase 2	Módulos Split + Extractor de cada modalidad funcionando de forma aislada (TF-IDF, SIFT, MFCC).	Texto / Imagen / Audio
3	Fase 2	Codebooks contruidos (top-k / K-Means) e índices invertidos (incluido SPIMI); persistencia en PostgreSQL.	Modalidades + Lead
4	Fase 2–3	Backend integrado con las 2 apps end-to-end; baselines GIN/GiST y pgvector implementados.	Backend + Lead + modalidades
5	Fase 4	Benchmarks completos (1K/10K/100K), métricas recogidas y gráficos comparativos generados.	Backend / Evaluación
6	Fase 4 + Entrega	Informe final en README/Wiki, demo ensayada, issues cerrados, presentación oral.	Todos

6. Estructura del repositorio en GitHub

Monorepo con un núcleo agnóstico (core/) y una carpeta por modalidad que implementa las interfaces comunes. Las comparativas nativas viven aparte, en baselines/.

```
multimodal-search/
├── README.md                # Informe tecnico principal
├── docker-compose.yml       # servicios: app + postgres (con pgvector)
├── .env.example             # variables de conexion (host, puerto, credenciales)
├── .gitignore
├── Makefile                 # make up / make ingest / make bench
├── requirements.txt
├── docker/
│   ├── postgres/
│   │   ├── Dockerfile
│   │   └── init/
│   │       ├── 01_extensions.sql # CREATE EXTENSION vector, pg_trgm
│   │       ├── 02_schema.sql    # chunks, codebooks, histogramas, metadatos
│   │       └── 03_indexes.sql   # GIN/GiST, HNSW/IVF
│   └── app/Dockerfile
├── src/
│   ├── core/                # ARQUITECTURA UNIFICADA (Lead)
│   │   ├── interfaces.py    # Splitter, Extractor, CodebookBuilder, InvertedIndex
│   │   ├── pipeline.py      # split -> extract -> codebook -> index
│   │   └── types.py
│   ├── text/                # (Ing. Texto)
│   │   ├── splitter.py      # parrafos
│   │   ├── extractor.py     # TF-IDF
│   │   ├── codebook.py      # tokenizar, stopwords, stemming, top-k
│   │   └── index/
│   │       ├── spimi.py     # OBLIGATORIO (enunciado)
│   │       └── bsbi.py      # OPCIONAL (baseline comparativo)
│   ├── image/               # (Ing. Imagenes)
│   │   ├── splitter.py      # patches
│   │   ├── extractor.py     # SIFT
│   │   ├── codebook.py      # K-Means -> visual words
│   │   └── index.py         # histogramas de visual words
│   ├── audio/               # (Ing. Audio)
│   │   ├── splitter.py      # ventanas deslizantes (100-200 ms)
│   │   ├── extractor.py     # MFCC
│   │   ├── codebook.py      # K-Means -> acoustic words
│   │   └── index.py
│   ├── db/
│   │   ├── connection.py    # cadena de conexion (lee del .env)
│   │   └── repositories.py  # acceso a codebooks/histogramas/metadatos
│   ├── baselines/           # COMPARATIVAS NATIVAS (Fase 3)
│   │   ├── gin_gist.py      # full-text de texto
│   │   └── pgvector.py      # HNSW/IVF para imagen y audio
│   ├── backend/             # (Backend/Apps)
│   │   ├── main.py          # FastAPI
│   │   ├── routes/
│   │   └── apps/
│   │       ├── visual_search.py # App 1
│   │       └── music_search.py  # App 2
│   └── eval/                 # (Evaluacion - Fase 4)
│       ├── benchmark.py     # cargas 1K / 10K / 100K
│       ├── metrics.py       # latencia, recall, memoria, I/O
│       └── plots.py

```

```

├── frontend/                # demo UI minima (suma en la presentacion)
├── data/
│   ├── raw/                # gitignored
│   └── README.md           # como descargar los datasets
├── scripts/
│   ├── download_data.sh
│   ├── ingest.py           # pipeline de carga completo
│   └── run_experiments.sh
├── results/                # tablas y graficos comparativos
├── docs/                   # fuente de la Wiki
└── tests/                  # tests por modulo

```

7. Convenciones de Git (esto se evalúa)

La rúbrica evalúa explícitamente la división de tareas, el cierre documentado de issues y la evidencia de colaboración. Para no perder esos puntos:

- **Ramas:** main (siempre desplegable) → develop → ramas por feature con prefijo de área (feat/text-spimi, feat/image-kmeans, fix/db-schema).
- **Pull Requests:** nadie hace merge directo a main; cada PR lo revisa al menos un compañero y se vincula al issue con “Closes #N”.
- **Issues con labels:** por área (texto, imagen, audio, backend, infra, eval) y por tipo (feature, bug, docs); asignados a una persona y a un milestone.
- **GitHub Projects:** tablero con columnas Backlog → En progreso → En revisión → Hecho y los milestones del cronograma. Es lo que el profesor revisará para verificar hitos a tiempo.
- **Commits:** convención tipo feat(text): implementar tokenización + stopwords.

8. Aclaraciones técnicas clave

8.1 SPIMI (obligatorio) vs. BSBI (opcional)

El enunciado pide implementar SPIMI para texto; BSBI no es requisito. SPIMI (Single-Pass In-Memory Indexing) construye diccionarios y posting lists por bloque sobre la marcha en memoria, sin ordenamiento global ni mapeo previo de termID, y vuelca a disco al llenarse. BSBI (Blocked Sort-Based Indexing) acumula pares (termID, docID), ordena por bloques y hace merge externo.

Implementar BSBI como baseline adicional es opcional pero suma a la Fase 4: permite comparar tiempo de construcción, accesos a disco y memoria entre ambos. Por eso en la estructura aparece src/text/index/ con spimi.py (obligatorio) y bsbi.py (opcional).

8.2 Docker: “base de datos lista en un comando”

El entregable incluye un Docker Compose con PostgreSQL + pgvector y scripts de inicialización. “Docker funcionando” significa que cualquier integrante clona el repo, ejecuta docker compose up, y obtiene PostgreSQL corriendo con la extensión pgvector, las tablas y los índices ya creados, sin instalar nada a mano. Esto elimina el clásico “en mi máquina sí funciona” entre 5 personas y distintos sistemas operativos, y es responsabilidad del Lead dejarlo listo en la Semana 1.

8.3 Conexión PostgreSQL ↔ Python

PostgreSQL (servidor) y el código Python (cliente) son piezas distintas que se comunican por red mediante una cadena de conexión (host, puerto, base, usuario, contraseña). Docker solo mantiene la base lista; la implementación en Python sigue siendo todo el trabajo real. Hay dos formas de desplegar, y el host de conexión cambia según cuál se use:

- **Opción A — Python local + Postgres en Docker:** el código corre en la máquina de cada quien y se conecta a `host=localhost`. La más cómoda para desarrollar y depurar.
- **Opción B — Python también en Docker:** ambos servicios en el mismo Compose; el host pasa a ser el nombre del servicio (`host=postgres`). La más reproducible para la entrega y la demo.

Patrón recomendado: desarrollar con la Opción A y entregar con la Opción B. Como el host se lee desde el `.env`, cambiar de una a otra no toca el código.

9. Entregables finales (checklist)

1. Repositorio GitHub con código fuente bien documentado.
2. Docker Compose: PostgreSQL + pgvector y scripts de inicialización.
3. Informe técnico conciso en README o Wiki: sistema y arquitectura, dataset, detalles por módulo, resultados experimentales (tablas y gráficos), análisis de trade-offs y conclusiones, e instrucciones de instalación y uso.
4. Al menos dos aplicaciones funcionando como casos de uso (App 1 + App 2).
5. Presentación oral con demostración en vivo de las dos aplicaciones.
6. GitHub Projects con planificación, milestones e issues cerrados como evidencia de seguimiento.